ItinerAI 项目搭建步骤 (基于Cursor开发)

📁 第1步：项目基础结构搭建

目标：创建项目骨架，让Cursor能够正确识别项目结构

代码块

```
itinerai-project/
├── README.md
├── .cursorrules
├── .gitignore
└── package.json  # 前端
```

验证：

1. 运行 `node -v` 检查Node.js环境
2. 运行 `npm init -y` 创建package.json
3. 检查 .cursorrules 是否被正确加载

📁 第2步：前端项目初始化

目标：创建React + TypeScript + Vite基础项目

代码块

```
frontend/
├── index.html
├── vite.config.ts
├── tsconfig.json
├── package.json
└── src/
    ├── main.tsx
    ├── App.tsx
    ├── App.css
    └── vite-env.d.ts
```

验证：

1. 运行 `npm run dev` 启动开发服务器
2. 访问 http://localhost:5173查看页面
3. 检查TypeScript编译无错误

📁 第3步：前端UI框架安装

目标：集成Ant Design和基础组件库

代码块

```
frontend/
├── src/
│   ├── components/
│   │   └── layout/
│   │       ├── MainLayout.tsx
│   │       └── LeftSidebar.tsx
│   ├── stores/
│   │   └── themeStore.ts
│   └── styles/
│       └── globals.css
```

验证：

1. 运行 `npm install antd @ant-design/icons`
2. 运行 `npm install zustand axios`
3. 创建MainLayout组件，验证Ant Design组件正常显示
4. 测试zustand状态管理是否能正常工作

📁 第4步：后端Python项目初始化

目标：搭建FastAPI基础环境

代码块

```
backend/
├── requirements.txt
├── pyproject.toml
├── .env.example
├── Dockerfile
└── app/
    ├── __init__.py
    ├── main.py
    └── core/
        ├── __init__.py
```

```
            └── config.py
```

验证：

1. 创建虚拟环境：`python -m venv venv`
2. 安装依赖：`pip install -r requirements.txt`
3. 运行：`uvicorn app.main:app --reload`
4. 访问 http://localhost:8000/docs查看Swagger文档

📁 第5步：数据库和ORM配置

目标：配置SQLAlchemy + PostgreSQL

代码块

```
backend/app/
├── models/
│   ├── __init__.py
│   ├── base.py
│   ├── user.py
│   └── itinerary.py
└── schemas/
    ├── __init__.py
    └── user.py
```

验证：

1. 创建PostgreSQL数据库
2. 运行数据库迁移：`alembic upgrade head`
3. 测试SQLAlchemy连接
4. 创建测试用户，验证CRUD操作

📁 第6步：核心业务模型实现

目标：实现用户模型和行程模型

代码块

```
# 验证步骤：
1. 创建User模型测试：
   - 注册API
   - 登录认证
   - API密钥生成和存储
```

```

2. 创建Itinerary模型测试:
   - 行程创建
   - 行程查询
   - 行程更新
   - JSONB字段存取
```

📁 第7步：API路由层实现

目标：实现基础API端点

代码块

```
backend/app/api/v1/endpoints/
├── __init__.py
├── auth.py
└── itinerary.py
```

验证：

1. 测试 `/api/v1/auth/register` 用户注册
2. 测试 `/api/v1/auth/login` 用户登录
3. 测试 `/api/v1/itinerary/create` 创建行程
4. 测试JWT令牌认证机制

📁 第8步：前端API客户端封装

目标：创建Axios实例和API调用函数

代码块

```
frontend/src/
├── api/
│   ├── client.ts
│   ├── auth.ts
│   └── itinerary.ts
└── stores/
    └── authStore.ts
```

验证：

1. 测试用户登录流程
2. 测试行程创建API调用

3. 验证请求拦截器和错误处理
4. 测试Token自动刷新机制

📁 第9步：前端主布局组件

目标：实现左右分栏布局

代码块

```
frontend/src/components/
├── layout/
│   ├── MainLayout.tsx
│   ├── Header.tsx
│   └── SiderMenu.tsx
├── itinerary/
│   └── ItineraryGraph.tsx
└── chat/
    └── ChatPanel.tsx
```

验证：

1. 布局响应式测试
2. 侧边栏折叠/展开功能
3. 路由导航测试
4. 组件间状态传递

📁 第10步：Redis和Celery配置

目标：配置异步任务队列

代码块

```
backend/
├── celery_app.py
├── app/tasks/
│   ├── __init__.py
│   └── generate_itinerary_task.py
└── app/core/
    └── celery_config.py
```

验证：

1. 启动Redis：`redis-server`
2. 启动Celery Worker：`celery -A celery_app worker --loglevel=info`

3. 测试简单的异步任务
4. 验证任务结果存储

📁 第11步：LangChain基础环境与工具配置

目标：安装LangChain并配置大模型连接（如Kimi）及基础工具（如搜索），为智能体开发做准备。

代码文件/操作:

1. 安装依赖: `pip install langchain langchain-community langchain-core`
2. 在 `backend/core/` 下创建 `llm.py`，配置LLM（如ChatOpenAI，指向Kimi API）。
3. 创建 `backend/core/tools.py`，配置 `DuckDuckGoSearchRun` 等搜索工具。
4. **验证**:
5. 运行 `python -c “from backend.core.llm import llm; print(llm.invoke(‘hello’))“`，测试LLM连接是否正常。
6. 测试搜索工具是否能返回实时结果。
7. **检查冲突**：确认新依赖未破坏现有API的启动。

第12步：实现“主控协调器(Orchestration Controller)”骨架

目标：实现智能体系统的调度中枢，管理用户会话状态和流程。

代码文件/操作:

1. 创建 `backend/agents/controller.py`。
2. 定义 `SessionState` 类（Pydantic模型），包含 `current_stage`，`user_nickname`，`travel_task_book`，`dossier`，`itinerary` 等字段。
3. 创建 `OrchestrationController` 类，包含 `state` 属性和 `route_to_agent()` 方法框架。
4. **验证**:
5. 能成功创建 `SessionState` 实例并赋值。
6. `OrchestrationController` 可被导入和实例化。
7. **检查冲突**：确保与现有的用户认证系统（`auth`）无耦合错误。

第13步：实现“需求分析师(Demand Analyst Agent)”智能体

目标：实现第一个智能体，能进行多轮对话并输出结构化 `旅行规划任务书`。

代码文件/操作:

1. 创建 `backend/agents/demand_analyst.py`。
2. 使用LangChain的 `create_react_agent` 或自定义 `AgentExecutor`，集成第11步配置的 `llm` 和 `search_tool`。

3. 将您提供的**需求分析师提示词(System Prompt)** 写入该模块。
4. 使用 `StructuredOutputParser` 确保输出格式为指定的JSON。
5. **验证**:
6. 独立测试该智能体：给定模拟用户输入（如“我想去上海玩3天”），能发起多轮询问，并最终输出格式正确的 `travel_task_book` JSON。
7. **检查冲突**：智能体运行不依赖未完成的 `情报搜集员` 等模块。

📦 第 14 步：定义 P0 核心数据结构（TravelTaskBook）

目标：统一前后端 & 智能体的结构化需求格式（你 PRD 里的固定选项框）**文件**：`backend/app/schemas/travel.py` **功能**：存储用户填写的所有需求（目的地、人数、天数、预算、节奏、偏好）**验证**：

1. 可导入 Pydantic 模型
2. 可赋值、可转 JSON
3. 与现有代码无冲突

📦 第 15 步：实现会话级本地记忆（Memory 模块）

目标：存储用户需求 + 选中景点，供智能体传递（最小化版不使用数据库）**文件**：`backend/app/core/memory.py` **使用**： `SessionMemory` 类，保存 / 读取 / 更新**验证**：

1. 存需求 → 能读出来
2. 刷新不丢失（会话级）
3. 不影响 FastAPI 启动

📦 第 16 步：优化 LangChain LLM / 工具配置

目标：支持地图 POI、天气、景点搜索（最小化可用版）**文件**：

- `backend/app/core/llm.py`
- `backend/app/core/tools.py` （新增地图 / POI / 天气模拟工具）**验证**：

1. LLM 可正常调用
2. 工具返回结构化数据
3. 无依赖冲突

📦 第 17 步：完善【需求分析师智能体】（替换对话 → 固定选项框）

目标：完全按你的 PRD：接收**固定表单**，不做对话采集**文件**：`backend/app/agents/demand_analyst.py` **功能**：

1. 接收前端表单 JSON
2. 输出标准 TravelTaskBook
3. 调用 API 获取目的地常规建议（天数 / 交通）**验证**：
4. 输入表单 → 输出正确结构
5. 可返回 “一键调整” 建议
6. 不依赖其他智能体

📦 第 18 步：实现【景点推荐模块】（P0 核心）

目标：按目的地 + 偏好返回 5-6 个卡片（含图片、介绍、风险提示）**文件**：`backend/app/services/spot_recommender.py` **功能**：

1. 调用地图 POI 获取景点
2. 小众地点自动补充书店 / 商场 / 展会
3. 输出卡片结构**验证**：
4. 输入需求 → 返回 5-6 个景点
5. 可显示预约 / 闭园标签
6. 可独立运行测试

📦 第 19 步：前端 → 需求表单组件（固定选项框）

文件：`frontend/src/components/itinerary/DemandForm.tsx` **功能**：

- 目的地
- 成人 / 儿童 / 老人
- 天数、预算、出行时间
- 旅行节奏、偏好**验证**：

1. 表单可填写
2. 提交后发送 JSON 给后端
3. 与布局无样式冲突

📦 第 20 步：前端 → 景点卡片多选组件

文件： `frontend/src/components/itinerary/SpotCardSelector.tsx` **功能：**

- 卡片展示（图 + 介绍 + 花费 + 提示）
- 多选
- 提交选中景点**验证：**

1. 接收后端景点列表正常渲染
2. 选中状态正确
3. 可提交选择到后端

📦 第 21 步：实现【行程规划师智能体】（最小化 5 步）

文件： `backend/app/agents/itinerary_planner.py` **功能（只做 P0）：**

1. 读取记忆里的需求 + 景点
2. 获取天气 + POI 坐标
3. 按日填充景点
4. 地理排序（耗时最短）
5. 预算对比提示**验证：**
6. 输入记忆 → 输出每日行程
7. 结构正确
8. 不依赖前端即可测试

📦 第 22 步：API 接口（P0 只做 4 个）

文件： `backend/app/api/v1/endpoints/itinerary.py` **接口：**

1. `/submit-demand` 提交需求
2. `/get-spots` 获取景点卡片
3. `/confirm-spots` 确认选择
4. `/generate-itinerary` 生成行程**验证：**
5. 可调用
6. 可返回数据

7. Swagger 可测试

📦 第 23 步：前端 → 行程展示组件（旅行图谱）

文件：`frontend/src/components/itinerary/ItineraryGraph.tsx` **功能**：

- 按天展示行程
- 显示景点、时长、花费、天气
- 显示预算提示**验证**：

1. 接收后端行程正常渲染
2. 样式不冲突
3. 可切换天数

📦 第 24 步：需求修改逻辑（分状态：生成前 / 生成后）

文件：

- 后端：`backend/app/agents/controller.py`
- 前端：`frontend/src/components/chat/ChatPanel.tsx` **功能**：
- 生成前：直接改需求
- 生成后：发指令 → 智能体修改行程**验证**：

1. 修改后记忆自动更新
2. 行程重新生成
3. 无状态错乱

📦 第 25 步：异常、加载、空状态（P0）

文件：前后端各加兜底**功能**：

- API 失败兜底
- 无景点时显示空状态
- 加载动画
- 表单校验**验证**：

1. 断网可提示
2. 无数据不崩溃

3. 流程可继续